

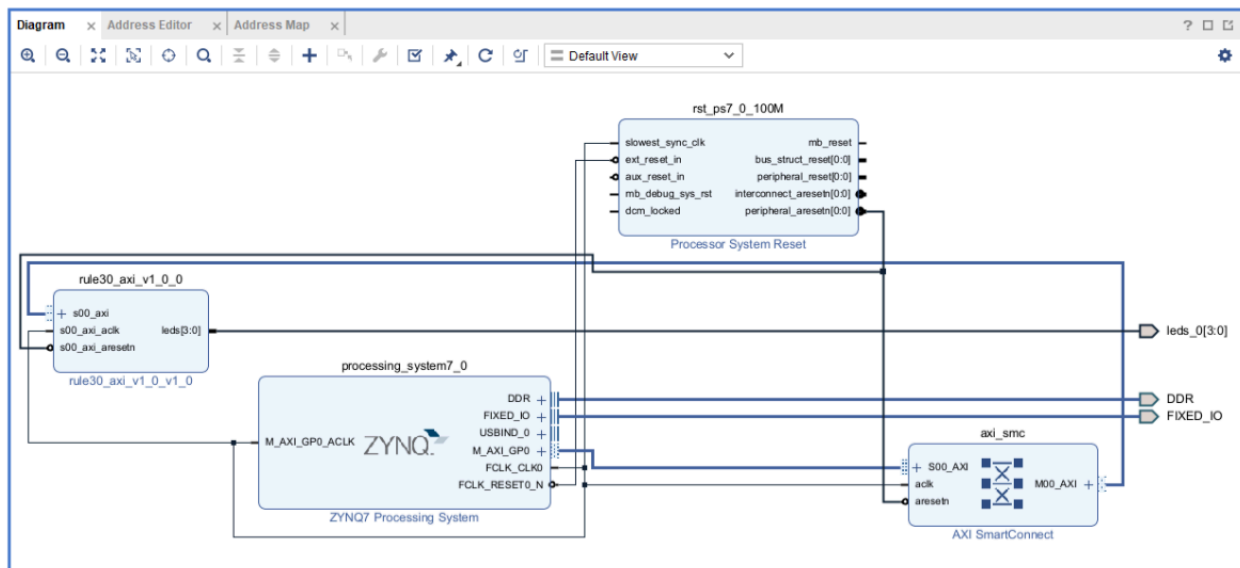
# Experiment 4 - Intermediate: Rule 30 Cellular Automation

Repository folder: [Experiment-4-Intermediate/](#)

## 1. Objective

The objective is to design a hardware-based pseudo-random number generator (PRNG) using the Rule 30 cellular automaton. This involves creating the Verilog RTL core, wrapping it in an AXI4-Lite interface, integrating it with the Zynq Processing System, and controlling it via a Python software driver to generate verified pseudo-random sequences.

## 2. Block Diagram



## 3. RTL Design

### 3.1 Module Hierarchy

```
rule30_axi_v1_0
|- rule30_axi_v1_0_S00_AXI
|- rule30_core
```

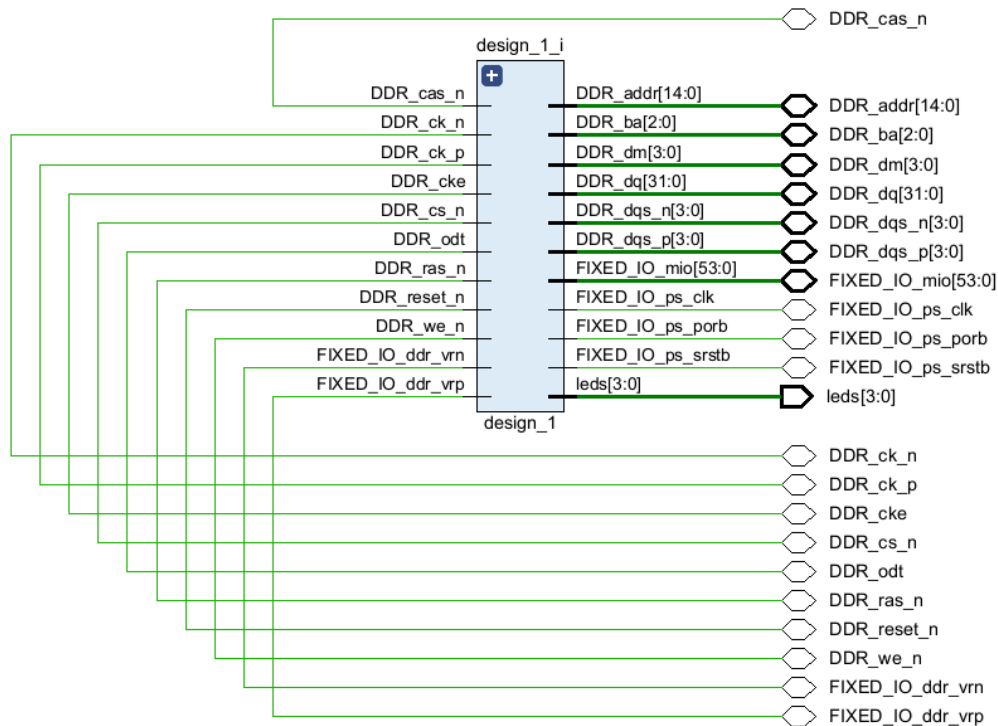
### 3.2 Module Descriptions

| Module                  | File                      | Purpose                                     | Key Inputs                     | Key Outputs        |
|-------------------------|---------------------------|---------------------------------------------|--------------------------------|--------------------|
| Rule30_axi_v1_0         | Rule30_axi_v1_0.v         | Top-level IP wrapper                        | S_AXI_*                        | S_AXI_*, leds[3:0] |
| rule30_axi_v1_0_S00_AXI | rule30_axi_v1_0_S00_AXI.v | AXI4-Lite memory-mapped register interface  | S_AXI_*                        | S_AXI_*, leds[3:0] |
| rule30_core.v           | rule30_core.v             | Implements Rule 30 cellular automaton logic | clk, rst_n, load_seed, seed_in | prng_out           |

### 3.3 Design Details

The rule30\_core is a parameterized sequential circuit without a complex FSM. On each clock cycle, the state updates using the Rule 30 combinatorial logic:  $\text{next\_state}[i] = \text{left\_bit} \oplus (\text{center\_bit} \mid \text{right\_bit})$ . The AXI wrapper maps address 0x00 (Write) to pulse load\_seed and write seed\_in. Address 0x04 (Read) retrieves the 32-bit prng\_out. The lower 4 bits of prng\_out are permanently assigned to the leds output port.

### 3.4 RTL Schematic



### 3.5 Constraints

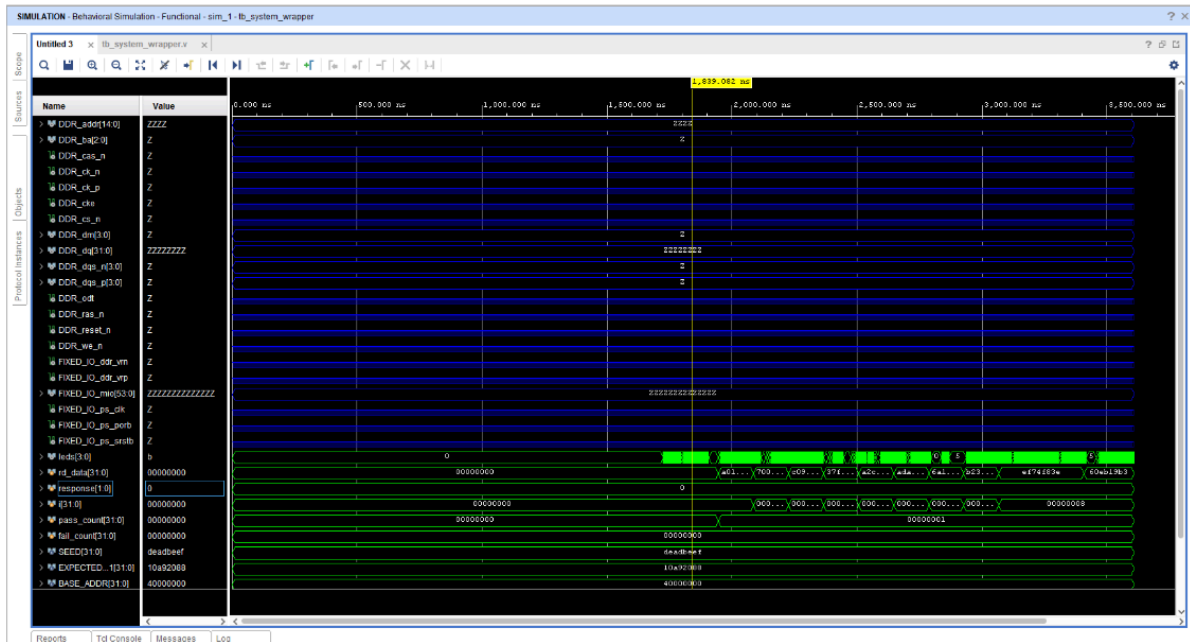
Pin mapping assigns the leds[3:0] output port to the 4 onboard LEDs (LD0-LD3) on the PYNQ-Z2 board. The clock constraint relies on the 100MHz FCLK\_CLK0 clock generated natively by the Zynq Processing System block.

## 4. Simulation Results

### 4.1 Testbench Description

tb\_rule30\_axi\_bfm.v uses an AXI Bus Functional Model to simulate processor transactions. It resets the system, writes the seed 0xDEADBEEF via the AXI write channel to address 0x00, and reads the generated PRNG value from address 0x04, checking for a match against a Python golden model.

### 4.2 Waveforms



The EXPECTED signal holds the golden-model first generation (0x10a92088). The pass\_count increments from 0 to 1, while the fail\_count remains steadily at 00000000 throughout the simulation. This proves the AXI read transactions were successful (response = 0 / OKAY) and the hardware generation exactly matches the expected mathematical model.

### 4.3 Testbench Output / Transcript

AXI BFM PASS: seed write to 0x00 and readback from 0x04 matched golden-model gen 1.

### 4.4 Result Summary

| Test Case                   | Expected   | Observed   | Status |
|-----------------------------|------------|------------|--------|
| AXI Seed Write and Readback | 0x10a92088 | 0x10a92088 | Pass   |

## 5. Hardware Implementation

### 5.1 Board Setup and Connections

[[IMAGE: Images/board\_setup.jpg]]

[[FILL: Which switches/buttons/LEDs/pins map to which signals; any external wiring.]]

## 5.2 Hardware Output

The PYNQ-Z2 board is powered and connected to the PC network. No external breadboard wiring is needed as the IP utilizes the 4 built-in LEDs for visual PRNG output.

## 5.3 Resource Utilization

| Name             | Slice LUTs<br>(53200) | Slice Registers<br>(106400) | Slice<br>(13300) | LUT as Logic<br>(53200) | LUT as Memory<br>(17400) | Bonded IOPADs<br>(130) | BUFGCTRL<br>(32) |
|------------------|-----------------------|-----------------------------|------------------|-------------------------|--------------------------|------------------------|------------------|
| system_wrapper   | 535                   | 672                         | 204              | 533                     | 2                        | 130                    | 1                |
| system_i(system) | 535                   | 672                         | 204              | 533                     | 2                        | 0                      | 1                |

## 6. Applications

1. Hardware acceleration for simulations and procedural gaming logic.
2. Cryptographic nonces in secure embedded SoC environments.
3. Fast noise generation for digital communication systems.

## 7. Conclusion

- Learning outcomes: Mastered AXI4-Lite interfacing, Verilog parameterized module design, Python driver test mocking, and Vivado block design workflows.
- Challenges faced and how they were solved: Syncing Vivado's auto-assigned base addresses with the Python software driver was resolved by dynamically parsing the .hwh (hardware handoff) file using the PYNQ framework.
- Future scope: Expand the IP to support AXI-Stream/Direct Memory Access (DMA) for high-throughput continuous random number delivery directly into DDR memory.

## 8. Demonstration Video